

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт з виконання лабораторної роботи

Реалізація смарт-контракту та оптимізація витрат гасу

Виконали студенти групи ФІ-52мн:
Волошин Ігор
Геращенко Володимир
Дорошенко Юрій

Київ 2026

Мета роботи

Отримання навичок роботи із смарт-контрактами або анонімними криптовалютами. Отримання практичних навичок роботи зі смарт-контрактами в екосистемі Ethereum, дослідження методів оптимізації витрат газу та розробка власного смарт-контракту на мові Solidity.

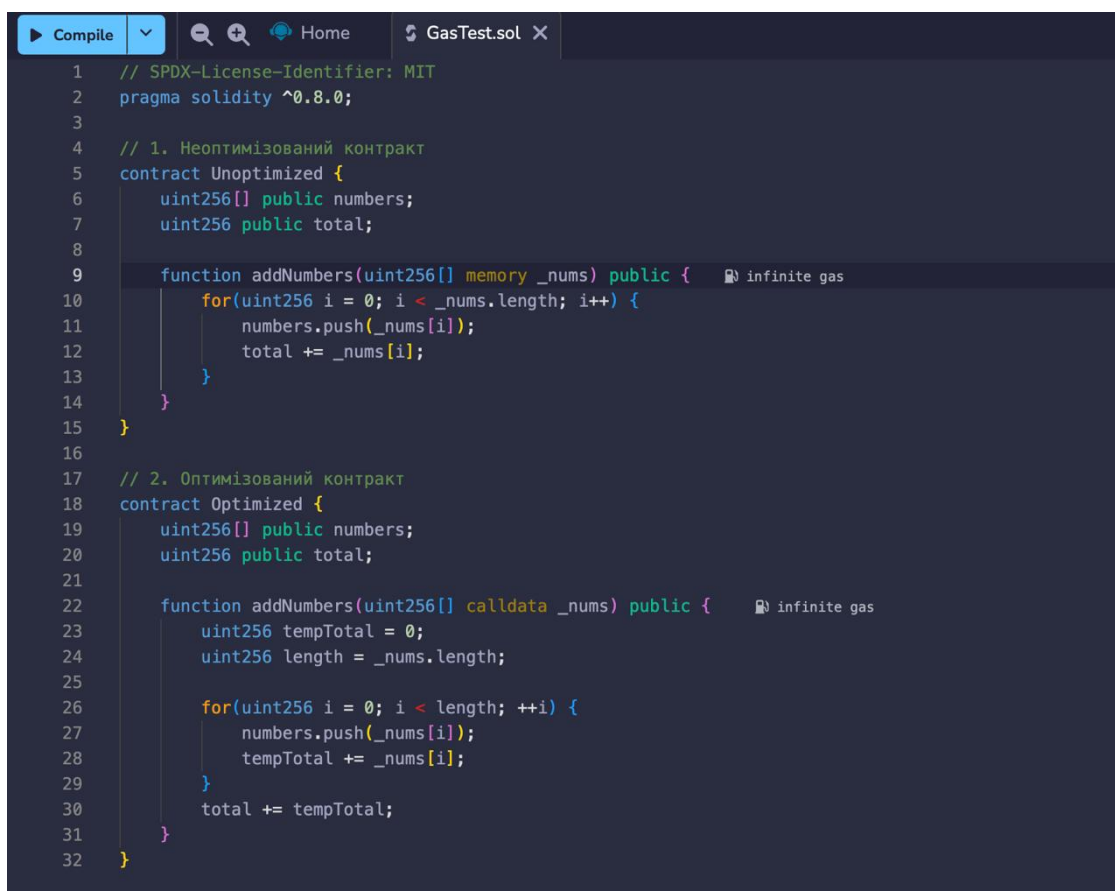
Завдання 1: Дослідження та оптимізація витрат газу

У сучасних блокчейн-мережах, таких як Ethereum, обчислювальні ресурси є обмеженими та платними. Кожна операція, що виконується в Ethereum Virtual Machine (EVM), має свою вартість, виражену в одиницях газу (Gas). Для розробника смарт-контрактів критично важливим є розуміння механізмів мінімізації витрат газу, оскільки це безпосередньо впливає на вартість взаємодії користувача з контрактом та масштабованість системи.

Ліміт газу (Gas Limit) запобігає нескінченним циклам, які могли б зупинити роботу мережі. Проте обмеження газу також створює ризики: якщо функція споживає занадто багато газу, вона може стати неможливою для виконання ("Out of gas error"), що фактично блокує кошти на контракті.

Для виконання роботи було обрано **Remix IDE** — інтегроване середовище розробки, яке дозволяє проводити налагодження контрактів у пісочниці Remix VM.

Для аналізу було використано два варіанти контракту для обробки масивів даних. Було реалізовано функцію, яка обробляє масив цілих чисел, зберігає їх у стейт та обчислює суму. Порівняно базову реалізацію з оптимізованою за методикою кешування даних у пам'яті.



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 // 1. Неоптимізований контракт
5 contract Unoptimized {
6     uint256[] public numbers;
7     uint256 public total;
8
9     function addNumbers(uint256[] memory _nums) public {
10         for(uint256 i = 0; i < _nums.length; i++) {
11             numbers.push(_nums[i]);
12             total += _nums[i];
13         }
14     }
15 }
16
17 // 2. Оптимізований контракт
18 contract Optimized {
19     uint256[] public numbers;
20     uint256 public total;
21
22     function addNumbers(uint256[] calldata _nums) public {
23         uint256 tempTotal = 0;
24         uint256 length = _nums.length;
25
26         for(uint256 i = 0; i < length; ++i) {
27             numbers.push(_nums[i]);
28             tempTotal += _nums[i];
29         }
30         total += tempTotal;
31     }
32 }
```

Рис. 1: Реалізація тестових контрактів

У неоптимізованому варіанті на кожній ітерації циклу виконувався запис у `storage`, що призводило до надмірних витрат. У оптимізованому варіанті було використано локальну змінну для акумуляції результату та ключове слово `calldata` для вхідних даних.

```

to Unoptimized.addNumbers(uint256[]) 0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8

transaction cost 299578 gas

execution cost 276834 gas

output 0x

decoded input {
  "uint256[] _nums": [
    "1",
    "2",
    "3",
    "4",
    "5",
    "6",
    "7",
    "8",
    "9",
    "10"
  ]
}

```

Рис. 2: Результат виконання неоптимізованого контракту

```

to Optimized.addNumbers(uint256[]) 0xd9145CCE52D386f254917e481eB44e9943F39138

transaction cost 295082 gas

execution cost 272338 gas

output 0x

decoded input {
  "uint256[] _nums": [
    "1",
    "2",
    "3",
    "4",
    "5",
    "6",
    "7",
    "8",
    "9",
    "10"
  ]
}

```

Рис. 3: Результат виконання оптимізованого контракту

Тип контракту	Execution Cost (gas)
Неоптимізований (Storage writes)	276 834
Оптимізований (Calldata & Caching)	272 338
Економія	4 496 (1.62%)

Табл. 1: Порівняння витрат газу

Завдання 2: Розробка власного смарт-контракту

Було розроблено смарт-контракт `SimpleBank`, який реалізує логіку прийому депозитів, зняття коштів та перевірки балансу з дотриманням сучасних стандартів безпеки (pattern Checks-Effects-Interactions).

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract SimpleBank {
5     mapping(address => uint256) private balances;
6     event Deposit(address indexed user, uint256 amount);
7     event Withdrawal(address indexed user, uint256 amount);
8
9     function deposit() public payable {
10         require(msg.value > 0, "Amount must be greater than 0");
11         balances[msg.sender] += msg.value;
12         emit Deposit(msg.sender, msg.value);
13     }
14
15     function withdraw(uint256 _amount) public {
16         require(balances[msg.sender] >= _amount, "Insufficient balance");
17         balances[msg.sender] -= _amount;
18         (bool success, ) = msg.sender.call{value: _amount}("");
19         require(success, "Transfer failed");
20
21         emit Withdrawal(msg.sender, _amount);
22     }
23 }
```

Рис. 4: Реалізація смарт-контракту

Було проведено успішне розгортання контракту в мережі Remix VM та виконано тестовий депозит сумою 1 ЕТН. Баланс контракту оновився коректно, що підтверджено Remix IDE.

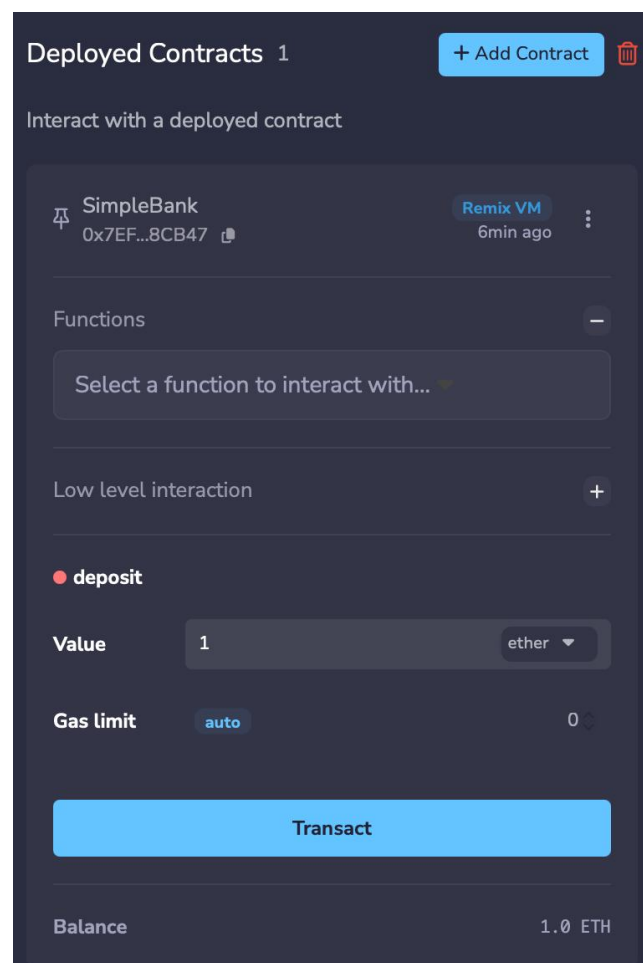


Рис. 5: Контракт на 1.0 ЕТН

Висновок

В ході лабораторної роботи було освоєно середовище Remix IDE та мову Solidity. Було досліджено внутрішні механізми споживання газу в мережі Ethereum. Отримані результати підтверджують, що архітектурні рішення на етапі написання смарт-контракту можуть суттєво впливати на його економічну ефективність. Експериментально підтверджено, що вибір типів пам'яті (Storage, Memory, Calldata) має критичний вплив на вартість транзакції. Використання ключового слова calldata та кешування змінних у локальну пам'ять дозволило скоротити витрати газу на виконання (Execution Cost) приблизно на 1.62% навіть для невеликого масиву з 10 елементів .

Оволодіння середовищем Remix IDE та мовою Solidity дозволило отримати навички розробки повного циклу — від написання коду та його компіляції до розгортання в імітованій мережі Remix VM та проведення тестових транзакцій. Розроблений контракт SimpleBank демонструє практичне застосування принципів безпечної розробки на Solidity.